

Inlämningsuppgift: IoT Telemetry & Analytics Platform

Magnus Ahlström - Systemmentor

Kurs: Dataplattformar, dataintegration och molnlösningar (30 YH-poäng)

Målgrupp: IoT- och embeddedutvecklare

Omfattning: Individuell inlämningsuppgift (Betyg: IG, G, VG)

Täcker kursmål: 1–10

1. Bakgrund & Scenario

Du arbetar som IoT- och molnutvecklare på företaget **Nordic IoT Solutions**. Företaget håller på att utveckla en ny plattform för att ta emot, lagra och analysera telemetridata (t.ex. temperatur, luftfuktighet och energiförbrukning) från uppkopplade givare och inbyggda system.

Din uppgift är att bygga och containerisera en robust, säker och skalbar mikrotjänstarkitektur för telemetrihantering. Systemet ska bestå av: 1. **REST API (Flask/Python)**: Ett webb-API för mottagning och hämtning av telemetridata, uppbyggt efter ett kontrakt i **OpenAPI 3.0**. 2. **Databas (PostgreSQL/MongoDB)**: En persistenslösning för lagring av sensordata. 3. **Säker Containerisering**: Optimerad Dockerfile med **multi-stage build** och körning med icke-root-behörighet (**USER 1000**). 4. **Orkestrering med Docker Compose**: Definierade tjänster med **nätverksisolering** (frontend och backend), **healthchecks** samt automatiserad **kontraktstestning** med **Schemathesis**. 5. **Teoretisk examination**: Flervalsformulär med 12 frågor kring GDPR, molnsäkerhet, arkitekturval och containerisering (**Teoretiska_Fragor.md**).

2. Kravspecifikation & Delmoment

Del 1: API & OpenAPI-kontrakt

- **OpenAPI Specifikation (openapi.yaml):**
 - Definiera följande ändpunkter:
 - * GET /health: Hälsö-check för container/orchestrator (returnerar {"status": "healthy"}).
 - * POST /api/v1/telemetry: Tar emot JSON-payload med `sensor_id` (string), `metric_type` (string), `value` (number) och `timestamp` (ISO 8601 string/date-time).
 - * GET /api/v1/telemetry: Hämtar alla eller filtrerade telemetriposter.
 - * GET /openapi.json: Exponerar OpenAPI-kontraktet i JSON-format för Schemathesis.
- **API-implementering (api/app.py):**
 - Implementera ändpunkterna i Python (Flask eller motsvarande).
 - Validera inkommande data mot specifikationen.

Del 2: Databas & Persistens

- Koppla API:et till en databas (PostgreSQL rekommenderas, men NoSQL/MongoDB är godkänt).
- Tabell/kollektion ska skapas automatiskt vid uppstart om den inte finns.
- Data ska persisteras med Docker-volymer så att mätvärden inte försvinner när containern startas om.

Del 3: Containerisering & Säkerhet (api/Dockerfile)

- **För Godkänt (G):**
 - En fungerande Dockerfile som installerar beroenden och startar API-applikationen.
- **För Väl Godkänt (VG):**
 - **Multi-stage Build:** Separera byggfas (`builder`) från körfas (`runtime`) för att minimera image-storleken och ta bort onödiga byggverktyg (t.ex. `gcc`).
 - **Icke-root användare:** Kör applikationen under en dedikerad icke-root användare (`USER 1000`).
 - **Renhet & Lageroptimering:** Rensa cacher (`--no-cache-dir`) och temporära filer.

Del 4: Orkestrering & Robusthet (docker-compose.yml)

- Definiera tjänsterna i `docker-compose.yml`:
 - `db`: Databastjänst med `volym-mount` för datapersistens.
 - `api`: REST API-tjänst.
 - `tests`: Kontraktstestare baserad på `schemathesis/schemathesis`.
- För Väl Godkänt (VG):
 - **Healthchecks**: Databasen ska ha en konfigurerad `healthcheck` (t.ex. `pg_isready`). API-tjänsten ska använda `depends_on: db: condition: service_healthy`.
 - **Nätverksisolering**:
 - * Nätverk `frontend`: Exponerar API till omvärlden/värddatorn.
 - * Nätverk `backend` (`internal: true`): Internt nätverk för kommunikation mellan API och databas. Databasen får **inte** vara direkt exponerad på `frontend`-nätverket.
 - **Automatiserad Kontraktstestning**: Tjänsten `tests` ska köra Schemathesis mot API:et (`http://api:5000/openapi.json`) utan kontraktsöverträdelser.

Del 5: Teoretisk del (Teoretiska_Fragor.md)

- Fyll i svarsblanketten för de 12 flervalsfrågorna (A, B, C eller D) i `Teoretiska_Fragor.md`.
- Frågorna täcker kursens teorimoment (GDPR, Shared Responsibility Model, kryptering, SOA, SQL vs NoSQL, Docker-säkerhet).

3. Förväntad Mappstruktur

Din inlämning ska ha följande struktur i ert Git-arkiv:

Inlämningsuppgift/	
Inlämningsuppgift.md	# Denna instruktion
Teoretiska_Fragor.md	# Dina teoretiska svar
Betygskriterier.md	# Betygsmatris för bedömning
starter-template/	# Din kod och konfiguration
openapi.yaml	# OpenAPI 3.0-specifikation
docker-compose.yml	# Docker Compose-konfiguration
README.md	# Instruktioner för hur din lösning byggs och körs
api/	
app.py	# Python API-kod

```
requirements.txt      # Python-beroenden  
Dockerfile            # Containerfil för API:et
```

4. Instruktioner för start och verifiering

För att verifiera att din lösning fungerar lokalt:

1. Navigera till `starter-template/`:

```
cd starter-template
```

2. Starta hela miljön och bygg containrarna:

```
docker-compose up --build -d
```

3. Kontrollera status på tjänsterna:

```
docker-compose ps
```

4. Kör kontraktstester med Schemathesis:

```
docker-compose run tests
```

5. Stäng av och städa upp:

```
docker-compose down -v
```

5. Inlämning & Bedömning

- Dela ditt repository med mig på github. Mitt användarnamn är Mahlstrom
- Inlämning sker via din angivna repository-länk / Lärplattformen.